SOC L1 LAB · INC-002

Attack Setup Guide: INC-002

Creating a rogue sudo account, phase by phase, inside the lab

INCIDENT	INC-002, Rogue local account added to sudo
REPORT DATE	2026-11-01
CLASSIFICATION	Lab exercise, sanitized for publication
ENVIRONMENT	Wazuh 4.14.7 · Debian 12 · QEMU/KVM
SEVERITY	High

Independent portfolio lab. No production systems, client data, or real credentials.

Contents

1. Authorization and safety
2. Topology and assumptions
3. Phase 0: Clean state
4. Phase 1: Start legitimate noise
5. Phase 2: Create the account
6. Phase 3: Set a password
7. Phase 4: Grant sudo
8. Phase 5: Verify
9. Phase 6: Capture evidence
10. Expected detections
11. Phase 7: Clean up
12. Variations for detection testing
13. Why this is realistic
14. Legal and ethical notes

A step-by-step script for reproducing INC-002 inside the isolated lab. It is for testing detection, not for use against systems you do not own.

---

1. Authorization and safety

Rule	Detail
Scope	Only the `soc-victim` VM on `10.0.2.0/24`
Never	Touch any other host, network, or account
Never	Use real credentials or employer systems
Authorization	You own the host and the VM
Reversibility	Snapshot before the run. Revert or clean up after
Data	Synthetic accounts and passwords only

The scenario models an insider or a compromised administrator session. It is deliberately an authorised admin action, because that is the case logs cannot distinguish from abuse without change-control evidence.

---

2. Topology and assumptions

```
Host 10.0.2.2
  ├── Wazuh manager / indexer / dashboard
  ├── analyst SSH session (key): 127.0.0.1:2222
  └── QEMU user network
        └── soc-victim 10.0.2.15
              sshd, auditd, FIM, wazuh-agent
              actor: analyst (uid 1000) via sudo
```

Assumptions:

- The base lab from `writeups/INC-001/02-reproduction-guide.md` is running.
- The agent is `Active`, auditd keys are loaded, `/etc` has realtime FIM.
- Rule 100121 is deployed and validated (see the INC-002 reproduction guide).
- `analyst` has passwordless sudo.
- No `svc-ops` account exists.

```
export LAB="$HOME/Documents/Eterna/soc-l1-lab"
export VMKEY="$LAB/lab/vm/keys/id_ed25519"
export VM="analyst@127.0.0.1"
export VMPORT=2222
```

---

3. Phase 0: Clean state

```
cd "$LAB/lab/vm"
./snapshot.sh save pre-inc002
sleep 5

ssh -i "$VMKEY" -p "$VMPORT" -o StrictHostKeyChecking=no "$VM" '
getent passwd svc-ops || echo "svc-ops absent"
getent group sudo'
```

Expected:

```
svc-ops absent
sudo:x:27:analyst,debian
```

If a validation account (`svc-test`) remains, remove it now:

```
ssh -i "$VMKEY" -p "$VMPORT" "$VM" '
sudo gpasswd -d svc-test sudo 2>/dev/null || true
sudo userdel -r svc-test 2>/dev/null || true'
```

---

4. Phase 1: Start legitimate noise

```
END=$(( $(date +%s) + 180 ))
setsid /tmp/noise.sh "$END" >/dev/null 2>&1 &
sleep 40
date -u +%FT%TZ > /tmp/inc002_start
```

If `/tmp/noise.sh` is missing, recreate it:

```
cat > /tmp/noise.sh <<EOF
#!/usr/bin/env bash
KEY=$VMKEY
END=\$1
while [ "\$(date +%s)" -lt "\$END" ]; do
  ssh -i "\$KEY" -p $VMPORT -o StrictHostKeyChecking=no \
      -o UserKnownHostsFile=/dev/null -o BatchMode=yes "$VM" \
      'uptime -p; systemctl is-active nginx' >/dev/null 2>&1
  ssh -i "\$KEY" -p $VMPORT -o StrictHostKeyChecking=no \
      -o UserKnownHostsFile=/dev/null -o BatchMode=yes "$VM" \
      'sudo -n systemctl is-active nginx' >/dev/null 2>&1
  sleep 30
done
```

```
EOF
chmod +x /tmp/noise.sh
```

---

5. Phase 2: Create the account

```
ssh -i "$VMKEY" -p "$VMPORT" -o StrictHostKeyChecking=no "$VM" '
sudo useradd -m -s /bin/bash svc-ops
getent passwd svc-ops'
```

Expected:

```
svc-ops:x:1003:1003::/home/svc-ops:/bin/bash
```

Why these flags:

Flag	Effect
`-m`	Creates `/home/svc-ops`
`-s /bin/bash`	Interactive shell, which a service account should not have

---

6. Phase 3: Set a password

```
ssh -i "$VMKEY" -p "$VMPORT" "$VM" '
echo "svc-ops:Summer2026!" | sudo chpasswd
sudo passwd -S svc-ops'
```

Expected: the account has a usable password (`P` in the second field).

This is part of the misconfiguration. Service accounts should use keys only, or no credentials at all if they are not interactive.

---

7. Phase 4: Grant sudo

```
ssh -i "$VMKEY" -p "$VMPORT" "$VM" '
sudo usermod -aG sudo svc-ops
id svc-ops
getent group sudo'
```

Expected:

```
uid=1003(svc-ops) gid=1003(svc-ops) groups=1003(svc-ops),27(sudo)
sudo:x:27:analyst,debian,svc-ops
```

This is the action the custom rule is designed to catch.

---

8. Phase 5: Verify

```
ssh -i "$VMKEY" -p "$VMPORT" "$VM" '
sudo -l -U svc-ops 2>&1 | tail -5'
```

If `sudo -l -U` requires a TTY, run it interactively or skip it. The group line from `getent` is sufficient evidence.

Record the time:

```
date -u +%FT%TZ > /tmp/inc002_action
echo "action: $(cat /tmp/inc002_action)"
```

Wait 60–90 seconds before extracting alerts.

---

9. Phase 6: Capture evidence

```
docker exec single-node-wazuh.manager-1 cat /var/ossec/logs/alerts/alerts.json \
  > /tmp/alerts_all.json

python3 - <<'PY'
import json, datetime
start = datetime.datetime.fromisoformat("2026-10-31T04:30:40+00:00")
end   = datetime.datetime.fromisoformat("2026-10-31T04:32:00+00:00")
rows=[]
for line in open('/tmp/alerts_all.json'):
    try: a=json.loads(line)
    except: continue
    if a.get('agent',{}).get('name')=='soc-victim' and \
       start <= datetime.datetime.fromisoformat(a['timestamp']) <= end:
        rows.append(a)
json.dump(rows, open('docs/evidence/INC-002-alerts.json','w'), indent=1)
print(len(rows), "alerts")
PY
```

Export the account records:

```
ssh -i "$VMKEY" -p "$VMPORT" "$VM" \
  'sudo journalctl --since "2026-10-31 04:30:40" --until "2026-10-31 04:31:20" --no-pager \
   | grep -Ei "useradd|usermod|chpasswd|new user|new group|sudo:"' \
```

```
  > docs/evidence/INC-002-account.log

ssh -i "$VMKEY" -p "$VMPORT" "$VM" \
  'getent group sudo; getent passwd svc-ops' \
  > docs/evidence/INC-002-membership.txt
```

---

10. Expected detections

Phase	Rule	Expected
`useradd`	80792	`Audit: Command: /usr/sbin/useradd`
`useradd`	5901 / 5902	New group and new user, level 8
`chpasswd`	80792	`Audit: Command: /usr/sbin/chpasswd`
`usermod -aG sudo`	100121	Privilege assignment, level 12, names `svc-ops`
File writes	550	`/etc/passwd`, `/etc/group`, `/etc/shadow`, `/etc/gshadow`
File writes	80781	Write access to `/etc/passwd` and `/etc/group`
Noise	5715 / 5402 / 80792	Routine logins, sudo, commands

Expected window volume: roughly 200–250 alerts, most of them audit command events.

---

11. Phase 7: Clean up

```
ssh -i "$VMKEY" -p "$VMPORT" "$VM" '
sudo gpasswd -d svc-ops sudo
sudo userdel -r svc-ops
getent group sudo
getent passwd svc-ops || echo "svc-ops removed"'
```

Expected:

```
sudo:x:27:analyst,debian
svc-ops removed
```

Or revert the snapshot:

```
cd "$LAB/lab/vm"
./snapshot.sh load pre-inc002
```

---

12. Variations for detection testing

12.1 `-G` instead of `-aG`

```
sudo usermod -G sudo svc-ops
```

This replaces the account's supplementary groups. Rule 100121 still matches, because `a2` is still `sudo`. Verify it fires, then restore the account's groups if needed.

The other spellings have their own rules: `usermod -a -G` to 100126, `gpasswd -a` to 100123, Debian `adduser` to 100124 (plus 100126 internally), and `useradd -G` to 100125.

12.2 A custom privileged group

On a RHEL-family host, the group is often `wheel`. The family (100121–100126) uses anchored PCRE2, `^(sudo|wheel|admin)$`. A different group needs the alternation changed or a CDB list. Keep the anchors: an unanchored list also matches `sudo-audit`. This is the most important variant to test. it is the difference between a portable rule and a Debian-only rule.

```
sudo groupadd -r ops-admin
sudo usermod -aG ops-admin svc-ops
# expected: no alert from 100121–100126 until the alternation or list is updated
```

12.3 Direct `/etc/group` edit

This simulates an attacker who can write the file without `usermod`:

```
sudo cp /etc/group /etc/group.bak
sudo sed -i 's/^\(sudo:x:27:.*\)$/\1,svc-ops/' /etc/group
```

Expected: FIM 550 on `/etc/group`, no alert from 100121–100126. This is the documented gap. A membership baseline or a FIM-diff rule is required to catch it. Restore the file afterwards:

```
sudo mv /etc/group.bak /etc/group
```

12.4 Account without a shell

```
sudo useradd -m -s /usr/sbin/nologin svc-svc
sudo usermod -aG sudo svc-svc
```

Rule 100121 still fires, correctly: the account is non-interactive but still privileged. This variant tests the policy "service accounts are never in sudo."

12.5 Add an SSH key

```
sudo -u svc-ops bash -c \
  'mkdir -p ~/.ssh; echo "ssh-ed25519 AAAATEST" >> ~/.ssh/authorized_keys'
```

Expected: rule 100120 from INC-001 fires on the key file, in addition to 100121. The combination is a strong signal: a new privileged account with a persistence mechanism.

---

13. Why this is realistic

- **The actor is an authenticated administrator**
  That is how most rogue local accounts are created, and why source IP alone cannot decide the case.
- **The name looks legitimate**
  `svc-ops` follows a service-account pattern.
- **The change is small**
  Two commands, under two seconds, in a queue of 237 alerts.
- **The privilege is implicit**
  Nothing in 5902 says "root". The analyst has to read the group membership.
- **The evidence is split**
  FIM shows the file change, audit shows the command, and the rule connects them.

---

14. Legal and ethical notes

- Run this only on the VM you created.
- The account, password, and UID are synthetic.
- Do not use these commands on an employer's system without written authorisation and a scoped engagement.
- Publishing the detection logic is fine. Publishing a real credential or a real hostname is not.